Introduction to ExpressJS?

ExpressJS is a robust framework built on top of Node.js. Now, why do we need a framework like Express when Node.js is already so powerful on its own? Well, while Node.js provides a great foundation for building server-side applications, it doesn't come with a lot of the features you need out of the box, such as routing, middleware support, or even handling HTTP requests and responses efficiently. This is where ExpressJS steps in.

- It is ExpressJS is a minimalistic framework. Now, what does that mean? In the world of web development, minimalism often refers to a framework that provides only the essentials, giving developers the freedom to build exactly what they need without unnecessary complexity. ExpressJS doesn't come with a lot of built-in features, but instead offers a simple yet powerful foundation on which you can build customized solutions.

  This minimalistic approach is one of the reasons why many developers prefer Express. It allows you to create lightweight applications that run efficiently, while still providing the flexibility to add only the features you need.

- **Streamlined Web Application Development:** ExpressJS is designed to streamline the process of web application development. Without a framework like Express, building a web server with Node.js would require manually handling HTTP requests, responses, routing, and other functionalities. While this is possible, it can quickly become tedious and prone to errors.

  Express simplifies this by offering a clean and intuitive API for handling these tasks. With Express, you can define routes, manage requests, and handle responses with just a few lines of code. This not only speeds up development but also makes your code more maintainable and easier to understand.

- **Foundation for Building Web Applications:** ExpressJS serves as an excellent foundation for building both simple and complex web applications. Whether you're working on a small personal project or a large enterprise-level application, Express provides the structure and flexibility needed to scale your application as it grows.

  Express is often described as the "backbone" of web applications built on Node.js. It doesn't impose any strict conventions or opinions on how your application should be structured, allowing you to design your app in a way that best suits your project's needs.

- **Robust Set of Tools:** Another compelling reason to choose ExpressJS is the robust set of tools it offers. These tools are designed to make your life as a developer easier. For example, Express comes with built-in middleware for serving static files, handling cookies, parsing request bodies, and more.

  Moreover, Express has a vibrant ecosystem of third-party middleware and plugins. This means that if Express doesn't have a feature you need, chances are there's a community-contributed middleware that does. This extensibility is what makes Express such a powerful and versatile tool for building web applications.

- **Simplifies Web Applications and APIs Development:** Finally, ExpressJS significantly simplifies the process of developing web applications and APIs. When building a RESTful API, for instance, Express makes it easy to define routes that respond to different HTTP methods, such as GET, POST, PUT, and DELETE.

  Express also supports various templating engines, making it simple to render dynamic content in your web applications. This is particularly useful for generating HTML pages on the server side, based on data retrieved from a database.

What is ExpressJS?

ExpressJS is a **minimalist web framework for Node.js**. But don't let the term "minimalist" mislead you—ExpressJS is incredibly robust and versatile. It simplifies the process of building web applications and APIs, making it a popular among developers.

Let's break it down:

1. **Core Purpose:**
   - ExpressJS is designed to help developers build web applications and APIs by providing a **flexible structure** for managing routes, handling requests and responses, and integrating middleware.
2. **Built on Node.js:**
   - At its core, ExpressJS leverages the **powerful Node.js platform**. Node.js itself is asynchronous and event-driven, which makes it highly efficient for handling multiple requests at once. ExpressJS builds on this foundation, enhancing Node.js with additional features to streamline the development of web applications.
3. **Flexibility and Efficiency:**
   - One of the reasons ExpressJS is so widely adopted is its **flexible architecture**. Whether you are building a simple single-page application or a complex multi-tiered system, ExpressJS allows you to structure your application according to your specific needs.
   - ExpressJS is also known for its **speed and efficiency**. Its lightweight design ensures quick execution times, which is crucial for applications that require fast response times and high throughput.
4. **Community and Ecosystem:**
   - The ExpressJS framework has a **vibrant and active community** of developers. This community not only contributes to the framework's continuous development but also provides a rich ecosystem of resources, libraries, and support. This makes it easier for developers to find solutions, learn best practices, and stay updated with the latest trends.

Key Features of ExpressJS:

"Express.js stands out due to several key features that streamline web development. Let's explore these in detail."

ExpressJS offers a wide range of features that empower developers to build robust and scalable web applications.

These features include routing, middleware, templating, static file serving, error handling, and more.

1. **Middleware:**
   - "Middleware functions are the backbone of Express.js. They are functions that have access to the request and response objects, and the next middleware function in the application's request-response cycle."
   - "Middleware can execute code, make changes to the request/response objects, end the request-response cycle, or call the next middleware in the stack."
   - "This modular approach allows for reusable code and cleaner application architecture."
2. **Routing:**
   - "Routing refers to how an application's endpoints (URIs) respond to client requests."
   - "Express provides a robust routing API that allows defining routes for various HTTP methods and URL patterns."
   - "This helps in organizing the application's functionality based on different endpoints."
3. **Templating:**
   - "Express integrates seamlessly with various templating engines like Pug, EJS, and Handlebars."
   - "These engines enable developers to generate dynamic HTML pages by embedding JavaScript code within templates."
   - "This is particularly useful for server-side rendering of views."
4. **Error Handling:**
   - "Express.js offers built-in error handling mechanisms."
   - "Custom error handlers can be defined to manage different types of errors gracefully."
   - "This ensures that the application can handle unexpected issues without crashing."
5. **Static File Serving:**
   - ExpressJS simplifies the process of serving static files such as images, CSS, and client-side JavaScript, making it convenient to build single-page applications.
6. **RESTful API Development:**
   - With its versatility, ExpressJS is an excellent choice for developing RESTful APIs. It allows developers to design APIs using the HTTP methods (GET, POST, PUT, DELETE) and supports JSON and XML data formats for communication.

Advantages of ExpressJS:

1. **Speed and Performance**: Being built on Node.js, ExpressJS inherits its performance capabilities. It is designed to be lightweight and, as a result, ensures fast and efficient handling of requests, making it an ideal choice for high-traffic applications.
2. **Large Community and Ecosystem**: ExpressJS enjoys a vast and active community of developers who continually contribute to the framework's growth. This active ecosystem means that there are numerous third-party modules, plugins, and middleware available to extend the functionality of your application.
3. **Flexibility and Scalability:** ExpressJS's minimalistic and unopinionated approach empowers developers to structure their applications as they see fit. This flexibility enables them to scale the application based on specific needs and handle complex projects effectively.
4. **Easy to Learn:** The simplicity of ExpressJS makes it relatively easy for developers to learn, especially those already familiar with JavaScript and Node.js.

ExpressJS Architecture:

Understanding the architecture of Express.js helps in grasping how it operates within a web application. Let's walk through the architecture diagram displayed on the slide."

1. **Node.js:**
   - "At the base, we have Node.js. It's the runtime environment that allows JavaScript to run on the server side."
   - "Node.js handles asynchronous operations and provides the core modules necessary for building server applications."
2. **HTTP Module:**
   - "Built on top of Node.js is the HTTP module. This module handles the low-level details of HTTP, such as creating servers and managing requests and responses."
   - "However, working directly with the HTTP module can be cumbersome for building complex applications."
3. **Connect:**
   - "Next is Connect, a middleware framework for Node.js. It provides a collection of middleware functions that can be used to handle various aspects of an HTTP request."

- "Express.js was originally built on top of Connect, inheriting its middleware capabilities."

4. **Express.js:**
   - "Finally, at the top layer, we have Express.js. It builds upon Node.js and Connect to provide a more streamlined and feature-rich framework."
   - "Express adds robust routing, middleware management, and other utilities that simplify the development process."

Installation - Express.js

Installing ExpressJS is a straightforward process using the Node Package Manager (NPM). NPM is a powerful tool that allows you to install, manage, and use packages of code developed by the Node.js community. To install ExpressJS, simply open your terminal or command prompt, navigate to your project directory, and run the following command:

- npm install express
- npm install –g express
- npm install express –save

npm install express	npm install express --save	npm install –g express
Installs the Express package locally in the project.	Installs the Express package locally and saves it to the dependencies section in the package.json file.	Installs the Express package globally.
Local Installation.	Local Installation with Dependency Management.	Global Installation
This command installs the express package and places it in the node_modules directory of the current project.	This command installs Express locally, just like npm install express, but with an additional step: it automatically adds Express to the dependencies section of the package.json file.	The package is installed in a global node_modules directory, typically located in your system's npm installation path.
Use npm install express for project-specific installations.	The --save flag was used to update package.json but is now redundant as npm does this automatically.	Use npm install -g express when you need to use Express globally or access its CLI tools.

This command will download and install ExpressJS into your project's dependencies, making it available for use in your application.

Command to check the installation locally or globally

```
D:\AcademicSubject\MCA_M.Sc.(IT)\MEAN-App>npm list express
MEAN-App@ D:\AcademicSubject\MCA_M.Sc.(IT)\MEAN-App
`-- express@4.18.2


D:\AcademicSubject\MCA_M.Sc.(IT)\MEAN-App>npm list -g express
C:\Users\CHARUSAT\AppData\Roaming\npm
`-- (empty)
```

ExpressJS Methods:

In web development, handling HTTP requests is a fundamental task. ExpressJS provides a streamlined way to define routes and handle these requests using various HTTP methods.

At the heart of web applications is the ability to handle HTTP requests and responses. ExpressJS provides a simple and intuitive mechanism for handling different HTTP methods, such as GET, POST, PUT, and DELETE. ExpressJS makes it easy for developers to access the request object, which contains information about the incoming request, such as headers, query parameters, and the request body. It also provides a response object, which allows you to send back responses to the client, including data, status codes, and headers.

Most commonly used methods in ExpressJS, which are essential for building RESTful APIs.

GET	Retrieves data from a server.
POST	Sends data to a server to create or update a resource.
PUT	Replaces an existing resource on the server with new data.
DELETE	Deletes a resource from the server.
PATCH	Partially updates an existing resource on the server.
OPTIONS	Gets information about the communication options available for a resource.

<u>Setting up an ExpressJS application</u>

```javascript
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello, ExpressJS!');
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});
```

Step 1: Importing Express

- "First, we import the Express module using the require function. This gives us access to Express's functionality by storing it in the express constant."

Step 2: Initializing the Express App

- "Next, we initialize an Express application by calling the express() function and storing the result in the app constant. This app will be used to define routes and handle HTTP requests."

Step 3: Handling a GET Request

- "Here, we define a route that listens for GET requests at the root URL '/'. When a request is received at this endpoint, the server responds with the message 'Hello, ExpressJS!'. This is done using the app.get() method, which takes two parameters: the route path and a callback function with req and res as its arguments."

Step 4: Setting the Port

- "The PORT constant is set to either the value of process.env.PORT or, if that is undefined, to 3000. The process.env.PORT variable allows us to specify a port dynamically, which is particularly useful in production environments. If no port is specified, the server will default to port 3000."

Step 5: Starting the Server

- "Finally, the server is started using the app.listen() method, which takes the port number as its first argument and a callback function as its second. When the server starts, it logs a message to the console indicating the port number on which the server is listening."

Express.js Request Object

- The request object in ExpressJS is a key part of handling incoming client requests.
- It contains information about the HTTP request, such as the request method, URL, headers, body, and more.
- It allows the server to process the request and respond accordingly.

Structure of the Request Object

- req.method: HTTP method used (GET, POST, etc.).
- req.url: The URL path of the request.
- req.headers: Contains all the headers sent by the client.
- req.params: URL route parameters (e.g., /user/:id).
- req.query: Query string parameters in the URL (e.g., ?name=John).
- req.body: The body of the request, often used for POST requests.

Accessing Request Parameters

- Route Parameter: Defined in route paths using the colon (:) syntax.

```
app.get('/user/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});
```

- Query Parameters: Can be accessed using req.query.
  URL example: /search?term=ExpressJS

```
app.get('/search', (req, res) => {
  res.send(`Search term: ${req.query.term}`);
});
```

Handling Request Body: Parsing the Request Body

- Body data is commonly sent in POST, PUT, and PATCH requests.
- Express provides middleware (express.json(), express.urlencoded()) to parse incoming JSON or URL-encoded data.
- Useful for handling form data or JSON payloads.

```
app.use(express.json());

app.post('/data', (req, res) => {
  res.send(`Received data: ${req.body.name}`);
});
```

Middleware in ExpressJS

- ExpressJS stands as a powerful and versatile framework for building robust applications.
- Its inherent flexibility is further enhanced by the concept of middlewares, which act as essential building blocks for **structuring and customizing the flow of requests and responses** within your application.
- Imagine middlewares as a series of checkpoints through which every incoming request must pass before reaching its destination.
- Each middleware performs a specific task, adding a layer of functionality or control to the request lifecycle.

What are Middlewares?

- **Functions:** Middlewares are functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. This allows them to modify the request, transform the response, or even terminate the request cycle altogether.
- **Intermediaries:** Think of middlewares as intermediaries between the client's request and the application's final response. They act as a bridge, enabling you to handle various tasks before the actual request handler gets invoked.
- **Chaining:** Middlewares can be chained together, forming a series of functions that execute in a defined order. This chain-like structure allows you to perform various operations on the request and response, creating a highly customizable workflow.
- Middleware functions are functions that execute during the request-response cycle. They can:
    - Execute any code.
    - Modify the request and response objects.
    - End the request-response cycle.
    - Call the next middleware function.

Middleware functions:

```
app.use(function (req, res, next) {
    // Middleware logic here...
    console.log('Time:', Date.now());
    next();
});
```

Types of ExpressJS Middleware's

- **Built-in Middleware:** These are middleware functions that come pre-packaged with ExpressJS. You don't need to install them separately, and they are often used to handle common tasks such as serving static files or parsing request bodies.
- **Third-party Middleware:** middleware functions developed by the community (or third-party developers) rather than being part of the core Express framework. These middleware packages extend the functionality of Express by **offering pre-built solutions for common tasks such as request logging, security, handling file uploads, and parsing request bodies**. Third-party middleware can be installed via npm (Node Package Manager) and integrated into your Express application using app.use() or directly in specific routes.
- **Custom Middleware:** middleware functions that are written by the developer to handle specific tasks in the request-response cycle. These functions are user-defined and serve purposes unique to the application, such as logging requests, authenticating users, validating input, or manipulating request and response objects.

Middleware in ExpressJS provides a mechanism for intercepting HTTP requests and responses, allowing you to add functionality and logic before they reach your application's core logic. Middleware enables you to perform tasks like authentication, authorization, logging, error handling, and more.

Understanding Body Parser Middleware in ExpressJS:

- Body Parser is middleware that parses incoming request bodies in a middleware before your handlers.
- It allows you to access data (usually from a form or JSON) submitted in a POST or PUT request via req.body.
- In Express 4.16+, body-parsing functionality is included in Express, so there's no need to install the body-parser package.
- HTTP requests can send data in various formats like JSON, form data, and raw data.
- By default, Express cannot read these types of request bodies.
- Body Parser allows Express to process these data types and makes the data available in req.body.
- Use Case: Handling form submissions or JSON data from APIs.

Types of Body Parser Middleware:

express.json( ): Parses incoming requests with JSON payloads.

- Required for applications that need to process JSON data, such as REST APIs.
- Allows easy access to JSON request bodies via req.body.
- It is essential for parsing data from client-side applications and APIs using JSON as the format.

```
const express = require('express');
const app = express();
app.use(express.json());  // Parse JSON data
app.post('/data', (req, res) => {
  const { name, age } = req.body;  // Access JSON data
  res.send(`Name: ${name}, Age: ${age}`);
});
```

- The `express.json()` middleware is a key part of handling JSON data in Express applications.
- Essential for modern web and mobile apps that exchange data via JSON.
- Built-in error handling for invalid JSON payloads.
- A simple and powerful tool for handling API requests with JSON format.

express.urlencoded(): Parses URL-encoded data (used for form submissions).

- express.urlencoded() is a built-in middleware function in Express.js.
- It parses incoming requests with URL-encoded payloads.
- Useful for handling form submissions.
- Based on the body-parser module, which is now part of Express.

```
app.use(express.urlencoded({ extended: true }));  // Parse URL-encoded data
app.post('/submit-form', (req, res) => {
  const { name, email } = req.body;  // Access form data
  res.send(`Form submitted! Name: ${name}, Email: ${email}`);
});
```

- extended: true: Allows for rich objects and arrays to be encoded in the URL-encoded format.
- extended: false: Use the simpler querystring library for parsing.
- The express.urlencoded() middleware is essential for handling URL-encoded form data.
- It simplifies accessing form submissions through req.body.
- The extended option gives flexibility in parsing complex objects and arrays.
- Efficient and simple for small payloads, especially for typical HTML form handling.

Serving Static Files middleware:

Why to use?

- Static files are essential for building the front-end of web applications.
- They include style sheets, client-side scripts, and media that enhance user experience.
- Serving static files from the server ensures faster page load and asset delivery.

How to serve static files:

- ExpressJS provides a built-in middleware function called express.static() for serving static files.
- This middleware allows you to serve files from a directory (like public or assets).
- Public Folder: Create a 'public' folder in your project's root directory to store static files like HTML, CSS, and images. This folder is usually used to store files that are directly served to the client.
- Express.static Middleware: Use the Express.static middleware to serve static files from the 'public' folder. It tells Express.js to look for files in that directory when a request for a static resource is made.
- Example:

  app.use(express.static('public'));

  The directory you specify (e.g., public) will contain your static files like CSS, JavaScript, images, etc.

Accessing Static Files in the Browser:

After starting the Express server, static files can be accessed directly by URL:

- **Example**:
  - Access index.html: http://localhost:3000/index.html
  - Access styles.css: http://localhost:3000/styles.css
  - Access script.js: http://localhost:3000/script.js
- Client-Side Access: The client browser can then access the static files through URLs like '/style.css' or '/images/logo.png'. The browser will automatically request and display the files from the server.